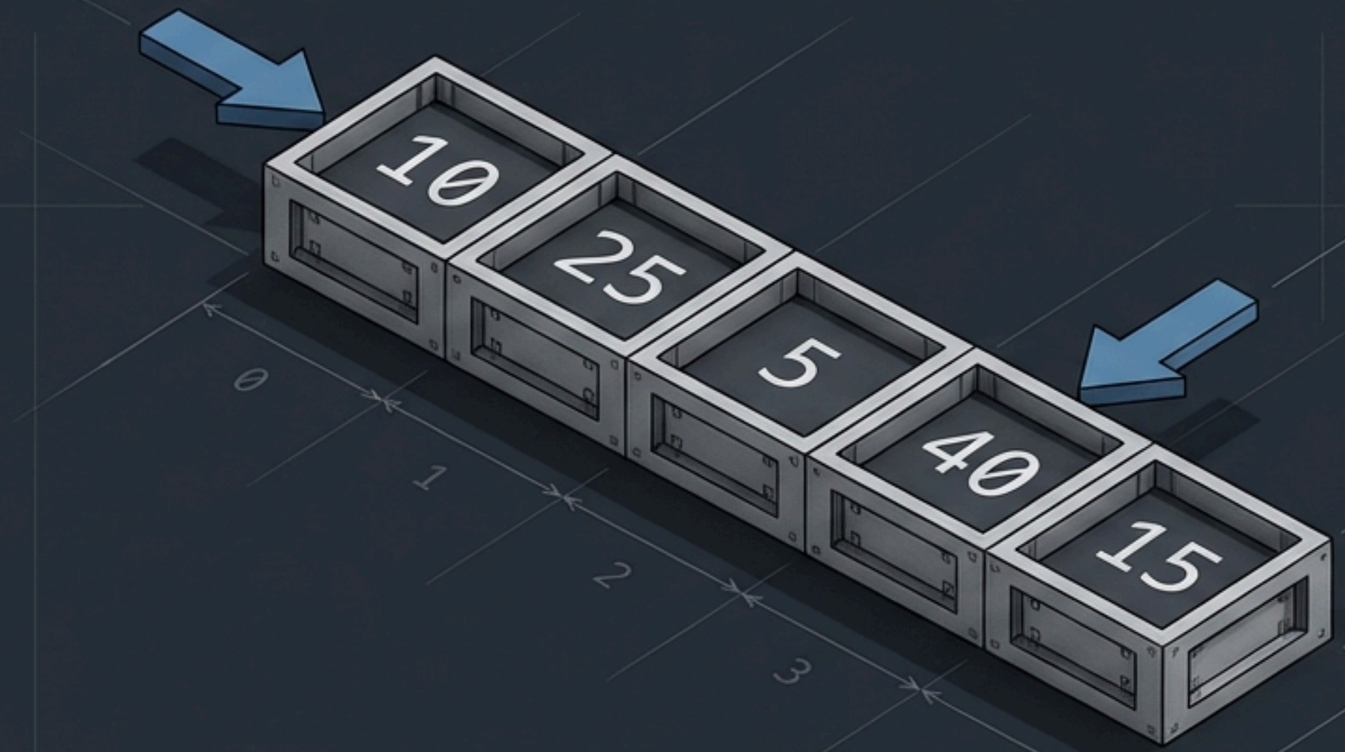
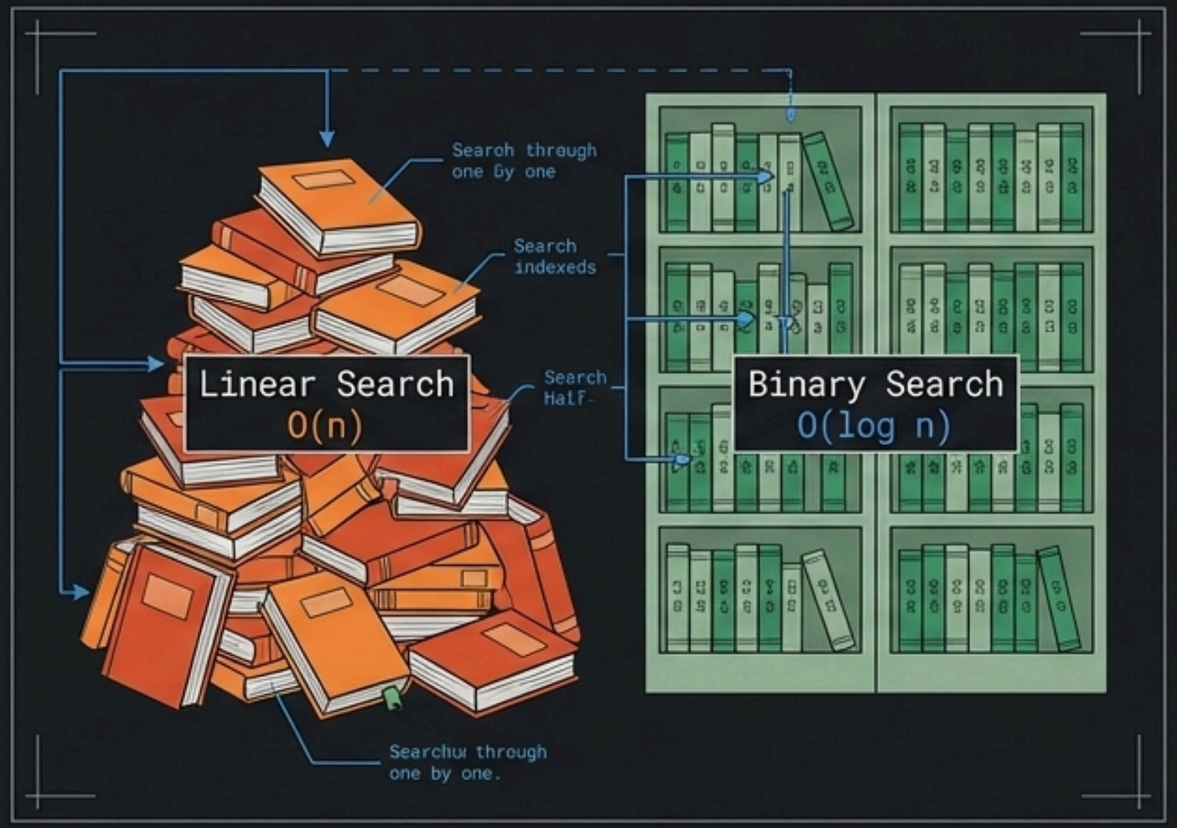
The DSA Interview Playbook

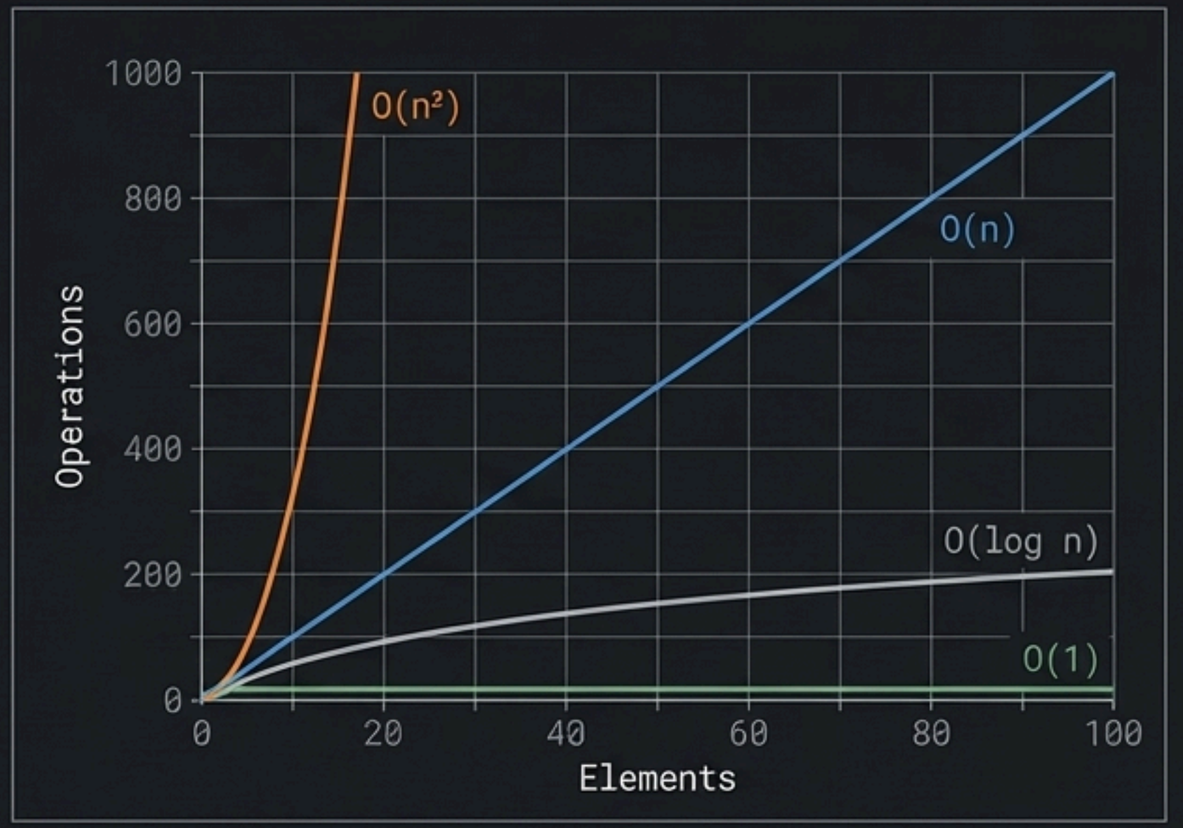
Mastering algorithmic patterns over problem memorization.



The Equation of Scale

Data Structure: How data is stored.
Algorithm: How data is processed.
The goal: Write algorithms with the smallest growth rate as input (n) scales.

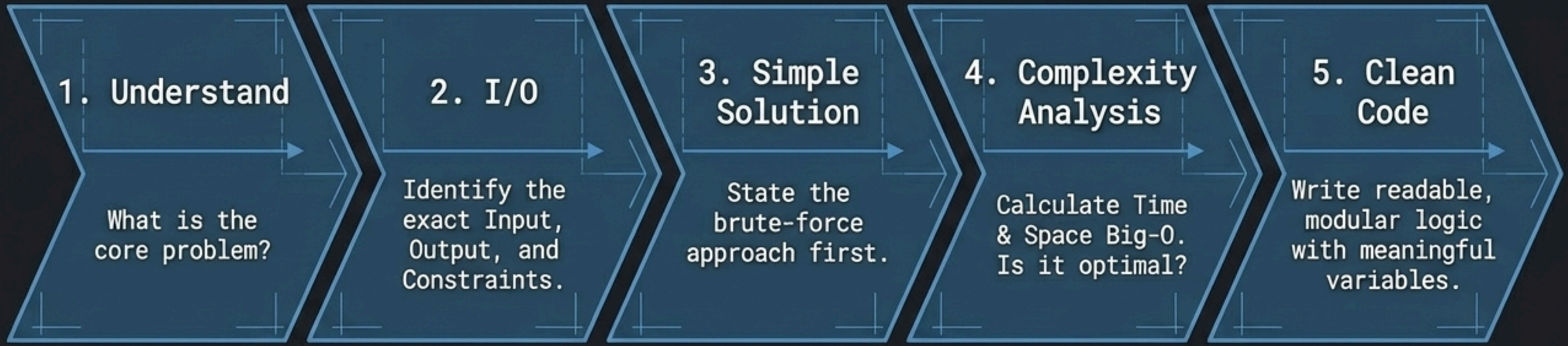




1 Loop = O(n) | 2 Nested Loops = O(n²) | Divide in Half = O(log n)

The Interviewer's Rubric

Interviewers test problem-solving ability, not just syntax.
They are looking for logical breakdown and optimization.



Golden Rule: First think. Then code.

Pattern Recognition Over Memorization

Arrays are the foundation of almost every DSA problem: fixed size, 0-indexed, O(1) access. You cannot memorize every array problem, but you can master the underlying architectural patterns.

1. Sliding Window
2. Two Pointers
3. Prefix Sum
4. Kadane's Algorithm
5. Binary Search
6. Hashing / Frequency Maps
7. Recursion

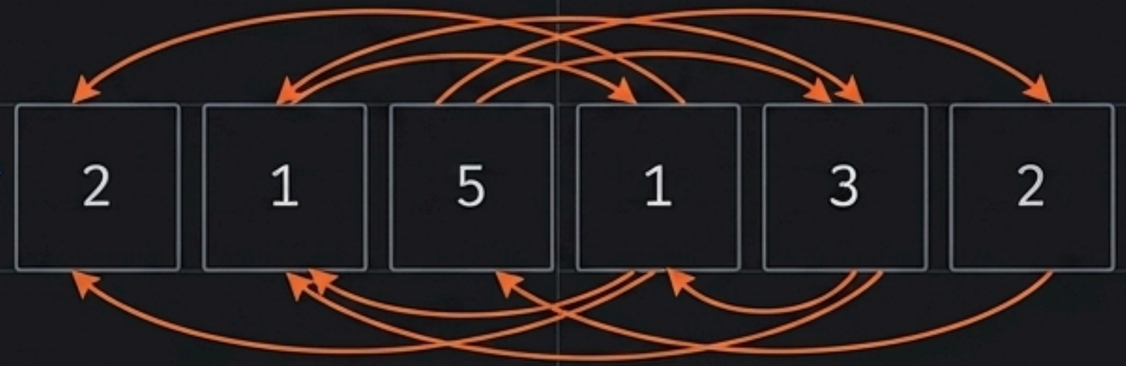
Pattern 1: Sliding Window

Use when: You need to process contiguous subarrays, substrings, or find a max/min in a fixed window size k.

The Logic: Reuse previous work instead of recalculating. Subtract the element leaving the window, add the element entering.

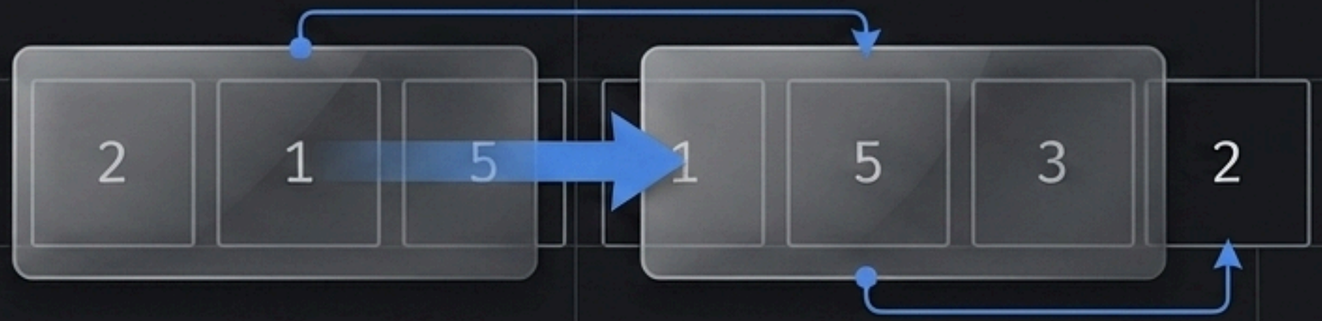
BRUTE FORCE

`O(n²) Recalculation`



OPTIMIZED

`O(n) Optimal`

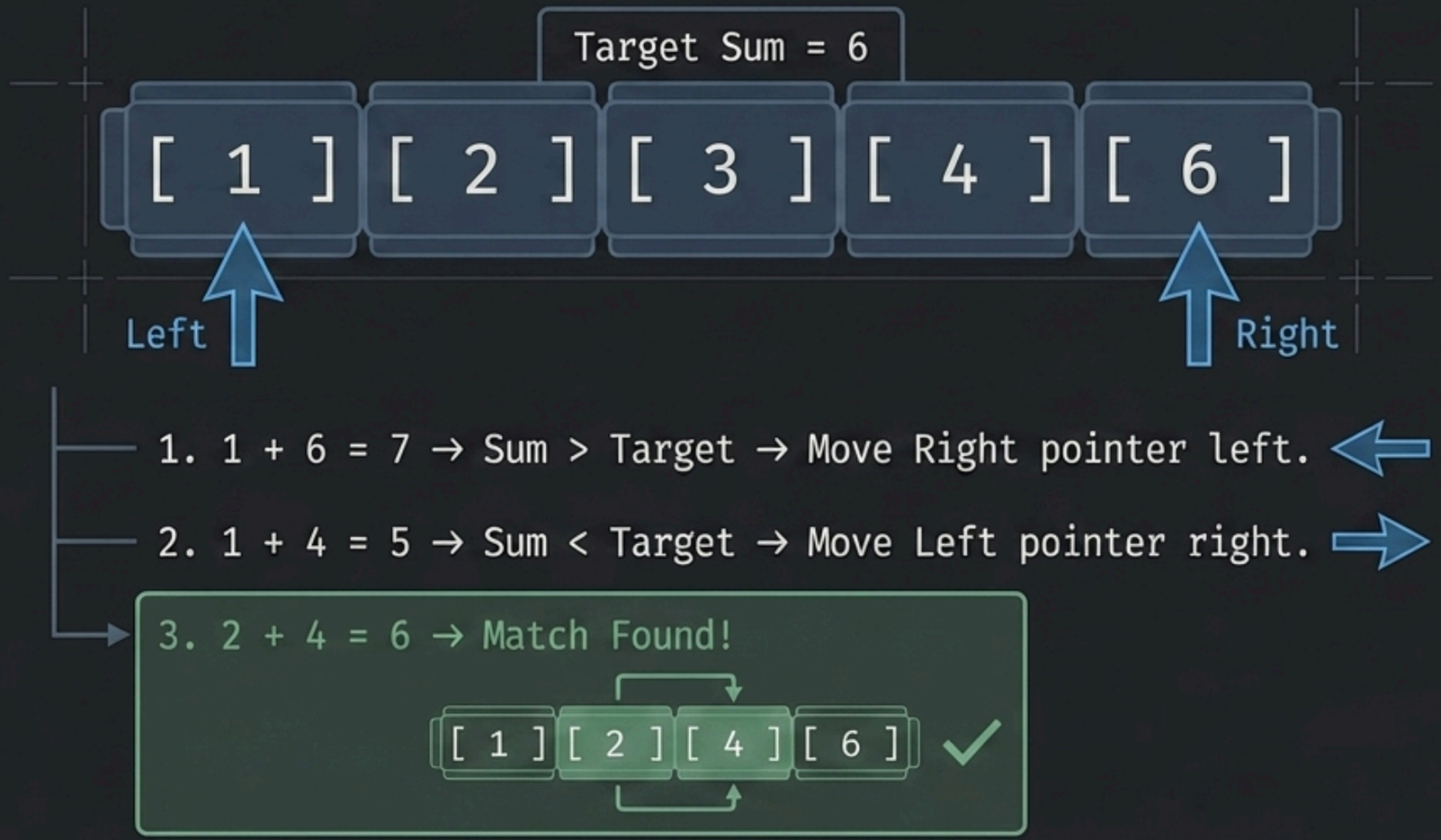



```
windowSum = windowSum - arr[i - k] + arr[i];
```

Pattern 2: Two Pointers

Use when: The array is sorted and you need to find pairs, target sums, or check palindromes.

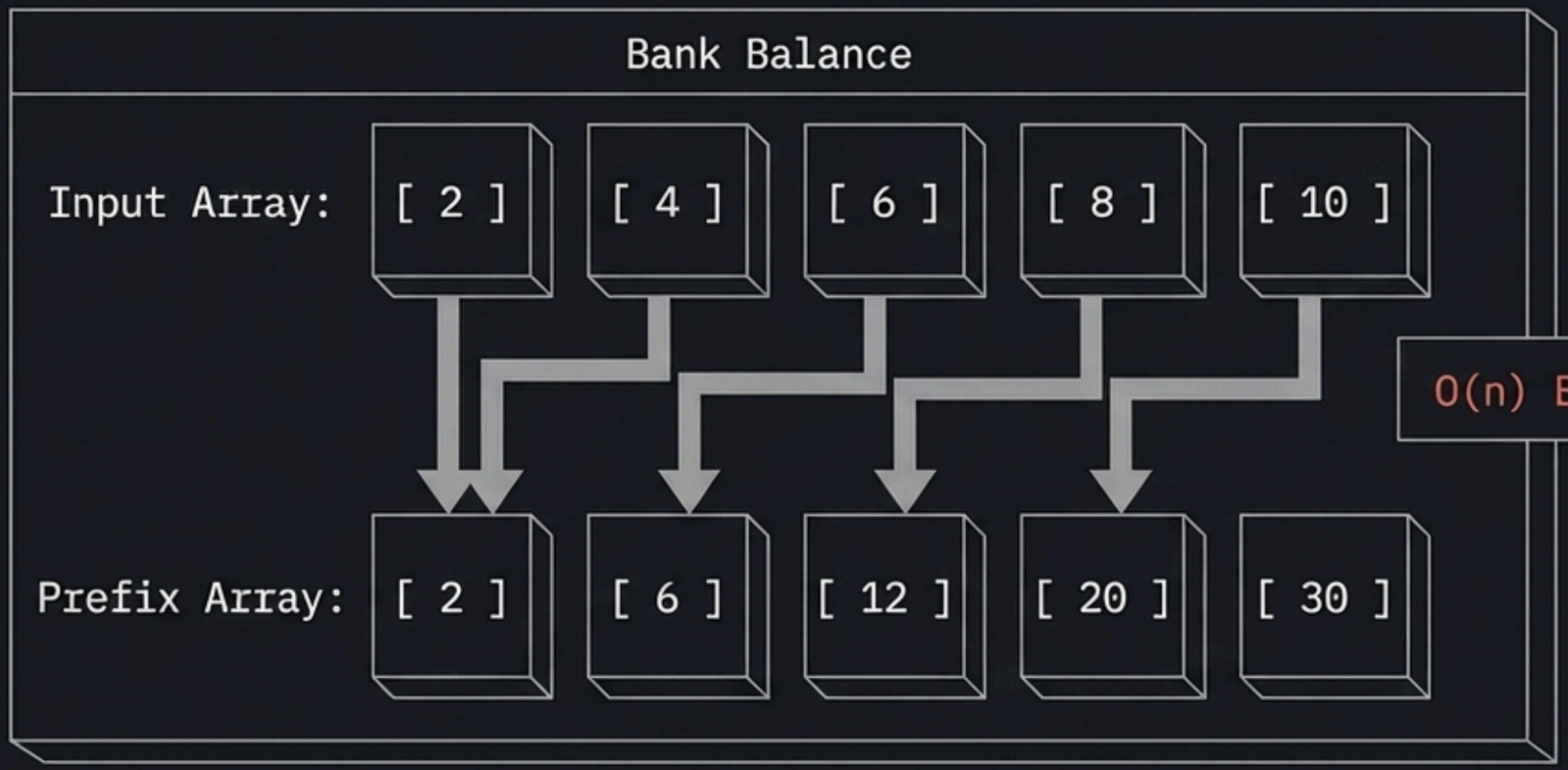
The Logic: Two indices move inward based on conditions, reducing nested loops into a single O(n) pass.

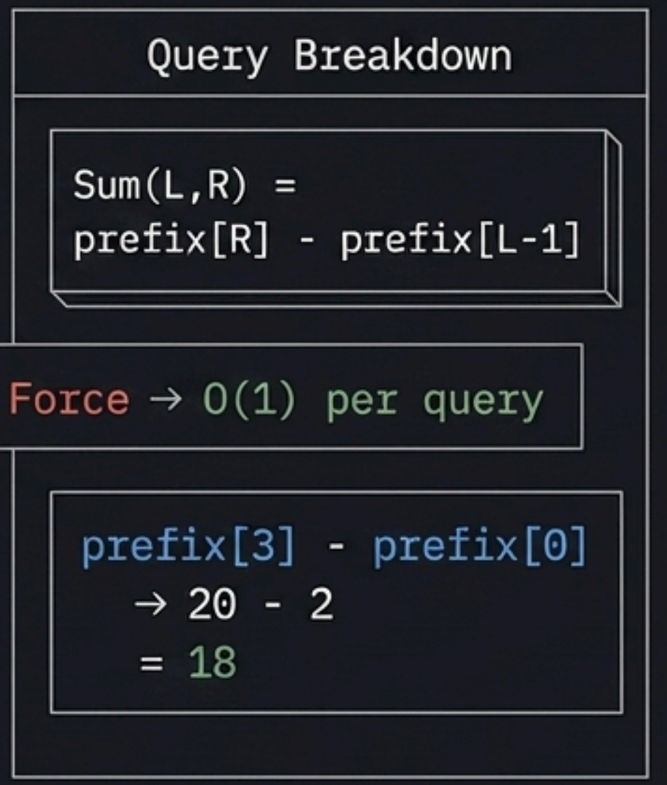


Pattern 3: Prefix Sum

Use when: You face multiple range sum queries, continuous math, or subarray calculations.

The Logic: Precompute cumulative sums once, answer range queries in constant time.

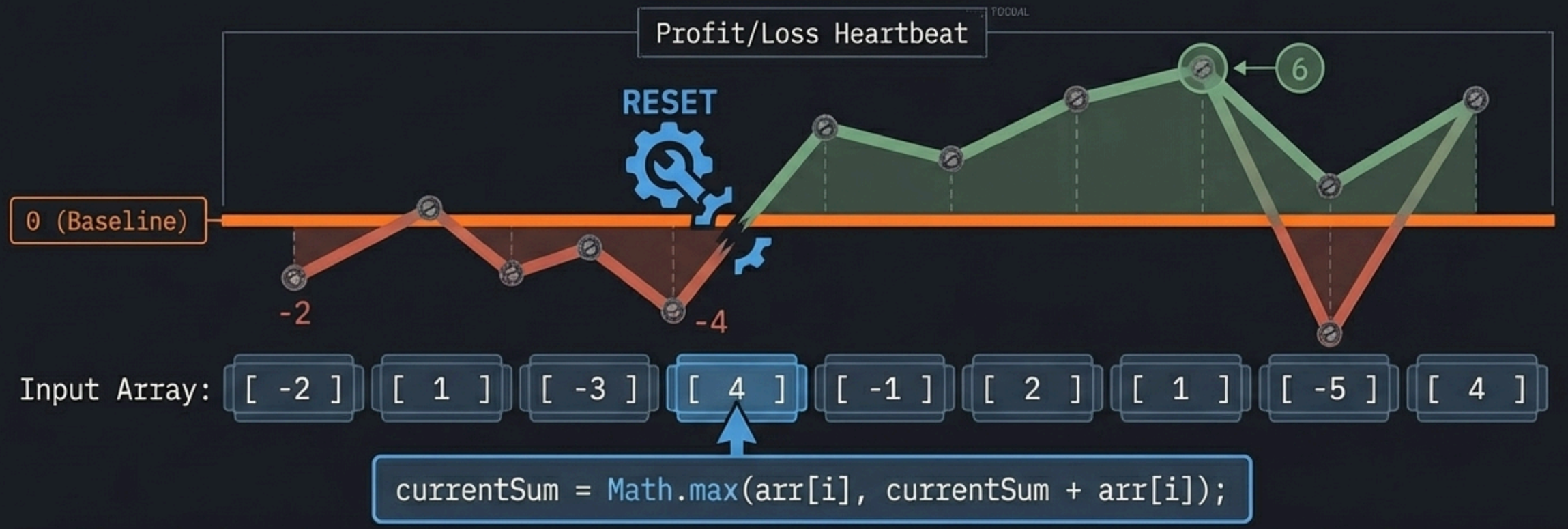




Pattern 4: Kadane's Algorithm

Use when: Finding the maximum contiguous subarray sum in an array containing both positive and negative numbers.

The Logic: Drop the negative, keep the positive. If the running sum drops below zero, reset it.

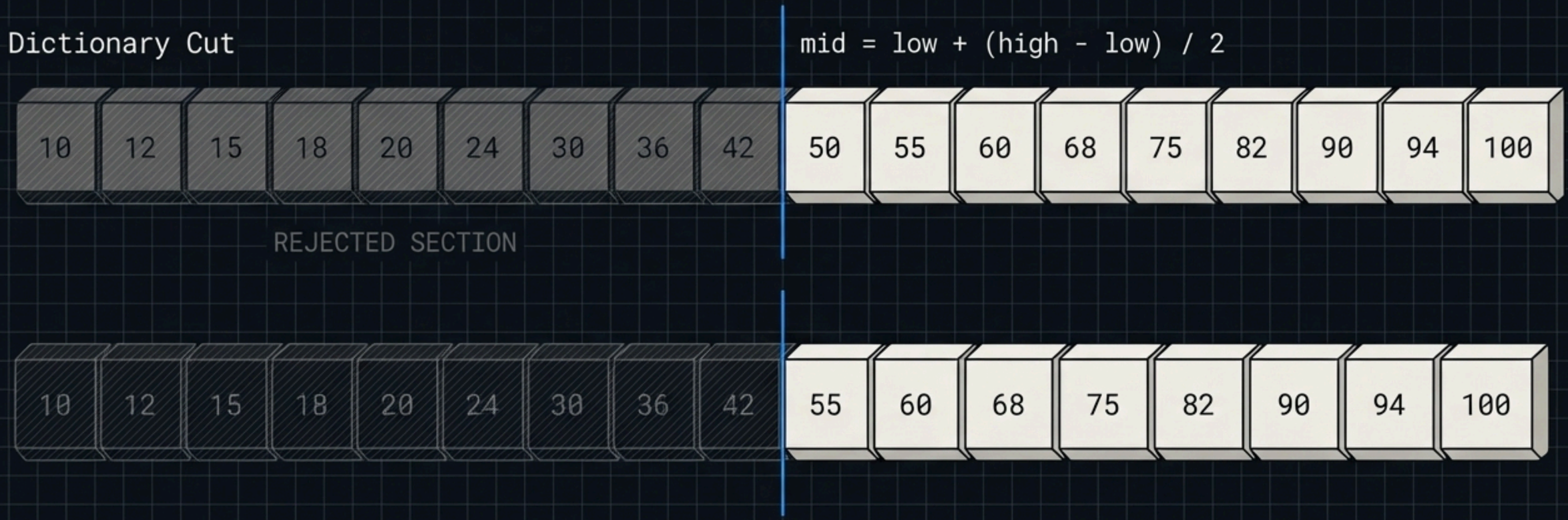


⚠ **Edge Case:** If all elements are negative, initialize maxSum = arr[0].

Pattern 5: Binary Search

Use when: Searching in a strictly sorted array or structure.

The Logic: Cut the problem in half every single time. 1,000,000 elements take just ~20 steps.

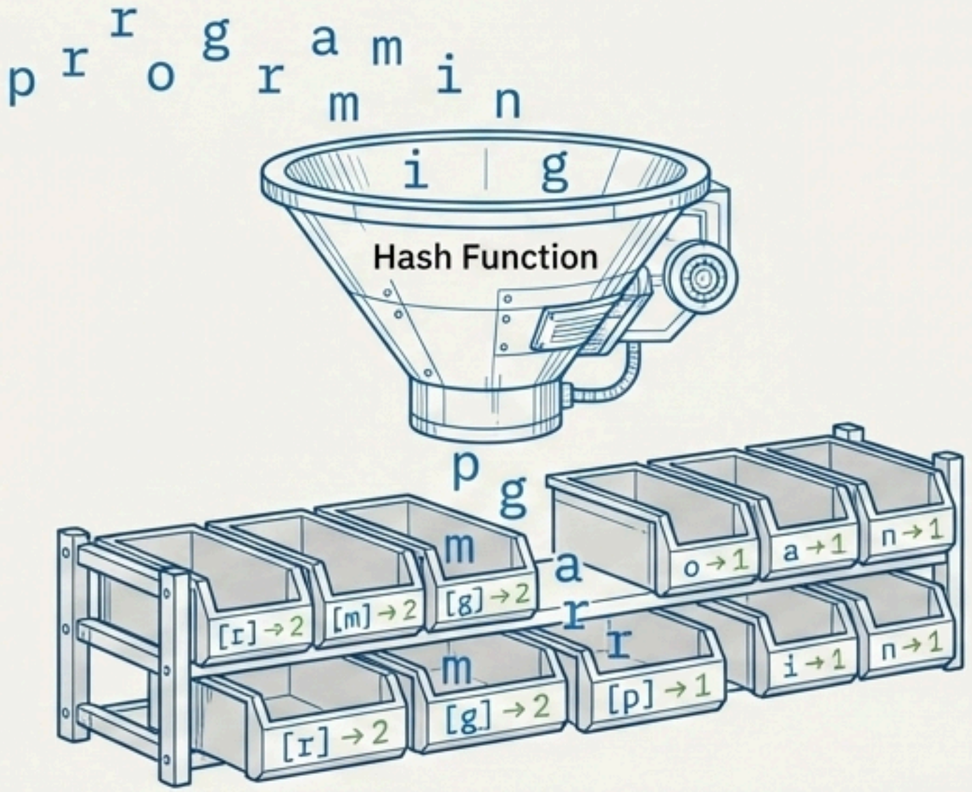


Linear Search O(n) → Binary Search O(log n)

Pattern 6: Frequency Maps & Hashing

Use when: Tracking occurrences, detecting duplicates, finding most frequent elements, or solving Anagrams.

The Logic: Count once, use many times. Transforms nested string comparisons O(n log n) into a single pass O(n) with O(1) lookups.

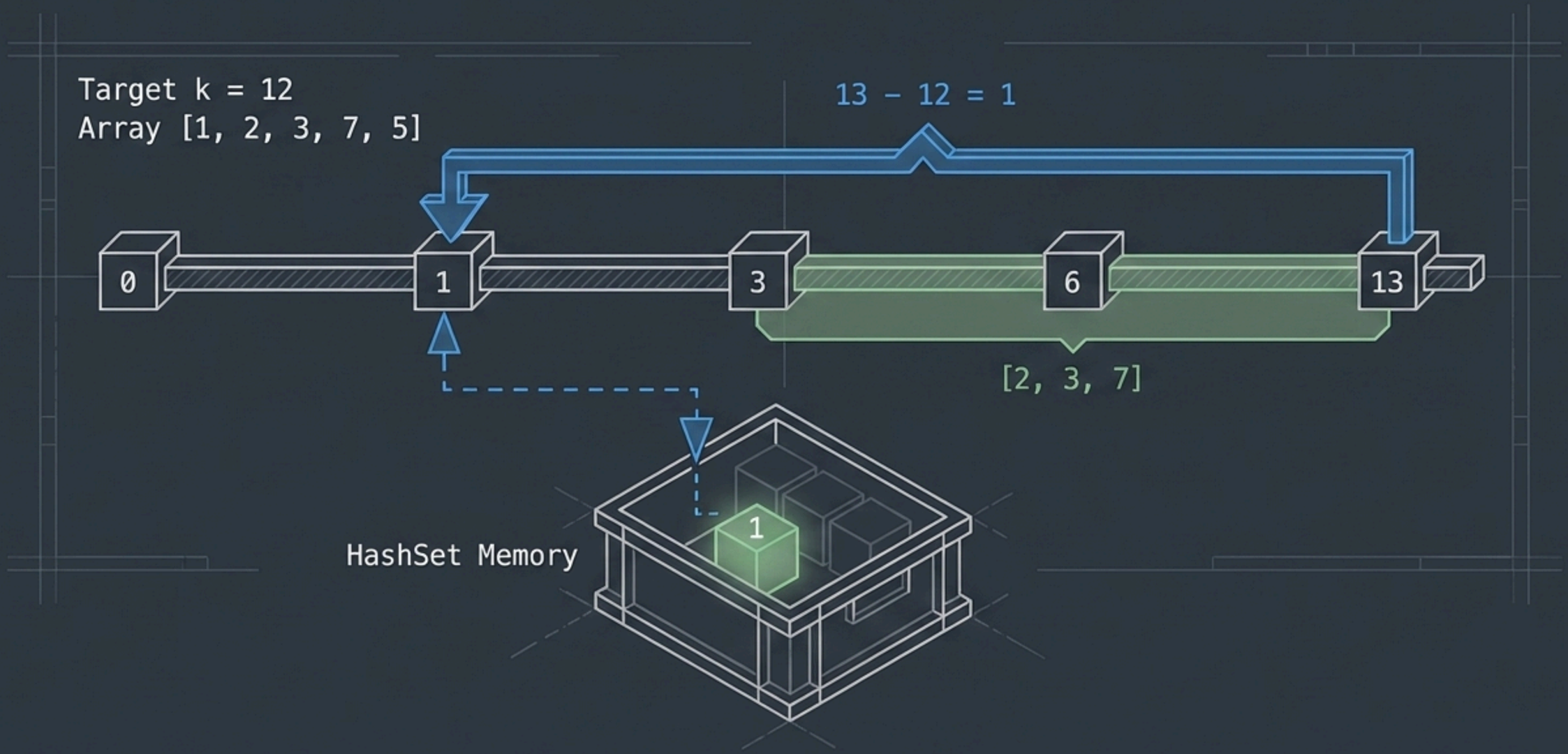



```
map.put(ch, map.getOrDefault(ch, 0) + 1);
```

Synthesis: The Subarray Sum = K Pattern

The Problem: Find if a contiguous subarray exists with a sum of k (including negative numbers). Sliding window fails here.

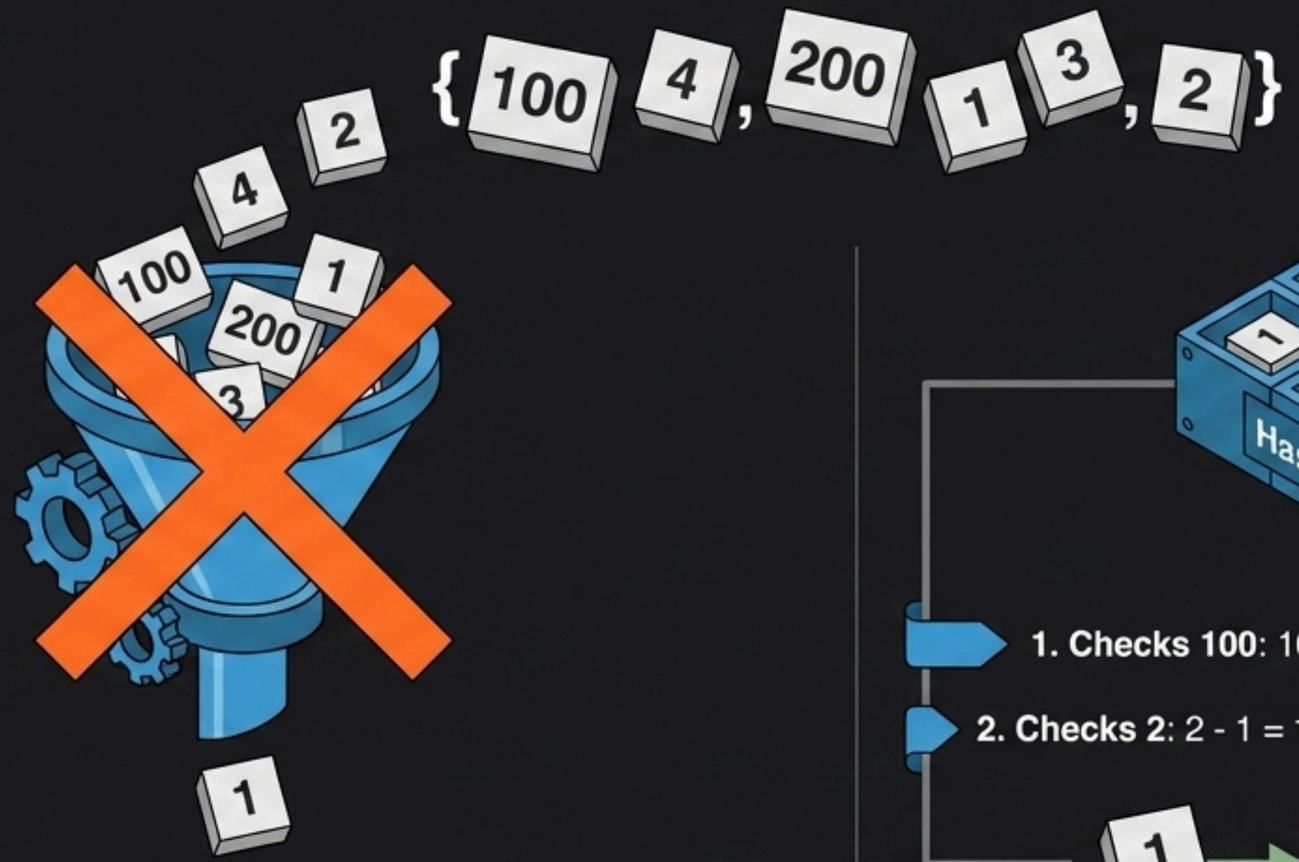
The Solution: Prefix Sum + HashSet. If "currentSum – k" exists in our history (HashSet), the subarray exists.



Pattern 7: Longest Consecutive Sequence

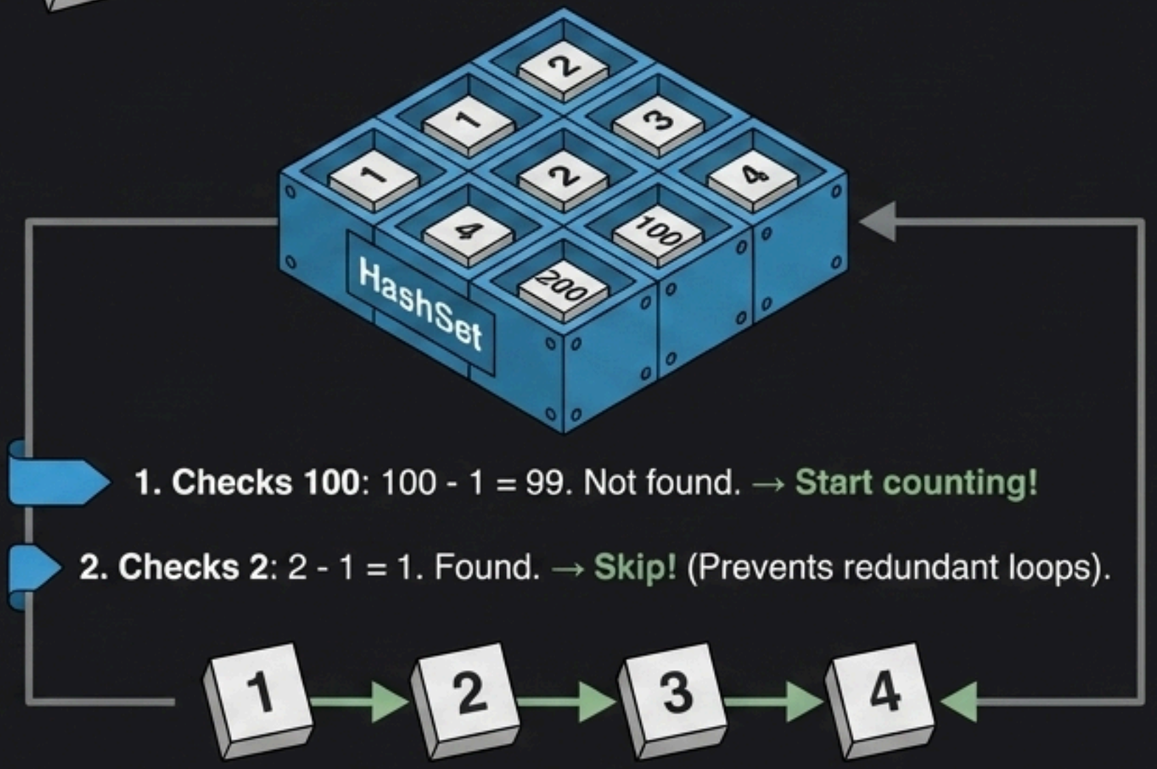
Use when: Finding continuous sequences in a completely unsorted array.

The Logic: Dump everything into a HashSet. Only start counting a sequence if it is the true beginning (num - 1 is NOT present).



Sorting the array first takes O(n log n)

Sorting the array first

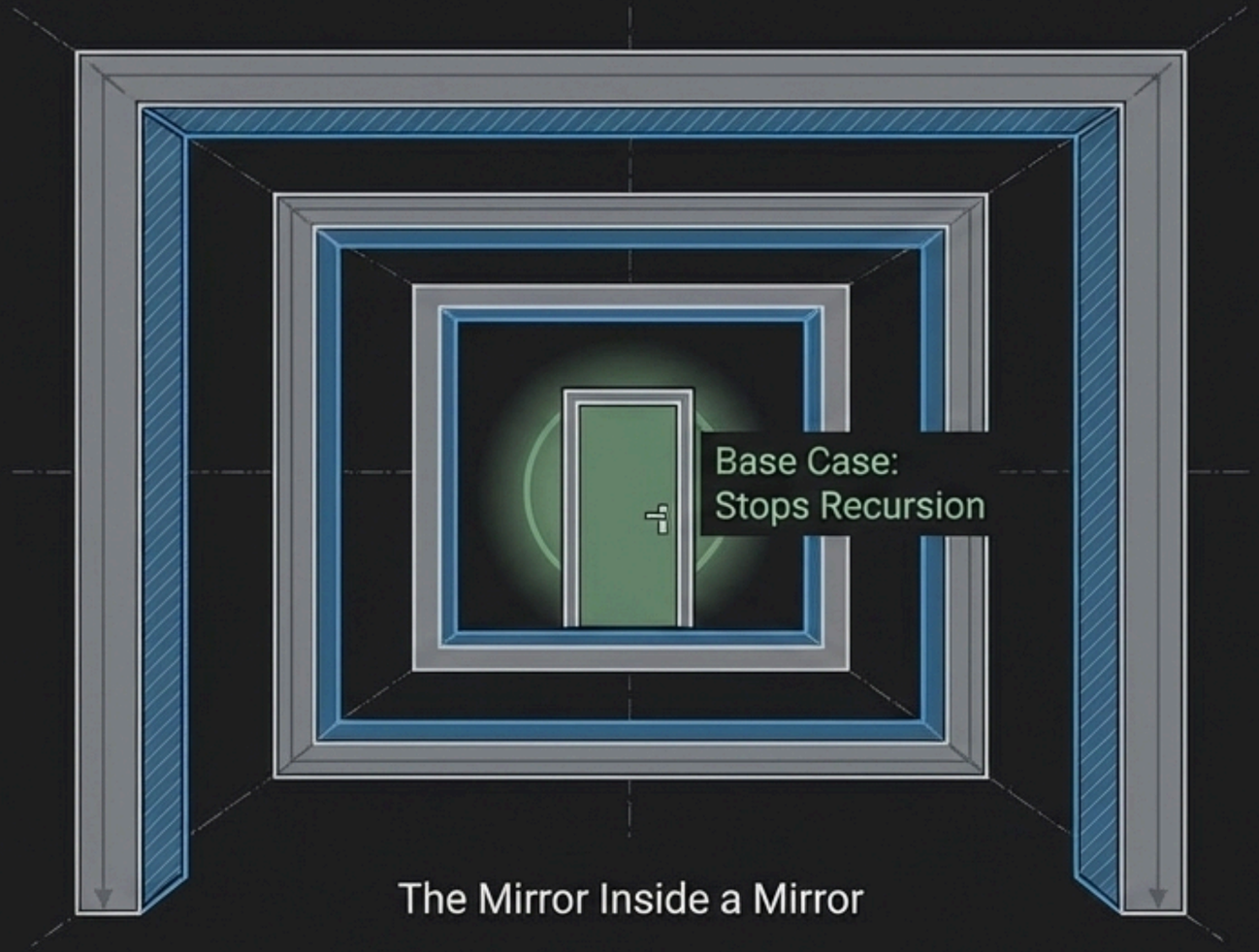


Highlights the O(n) chain:

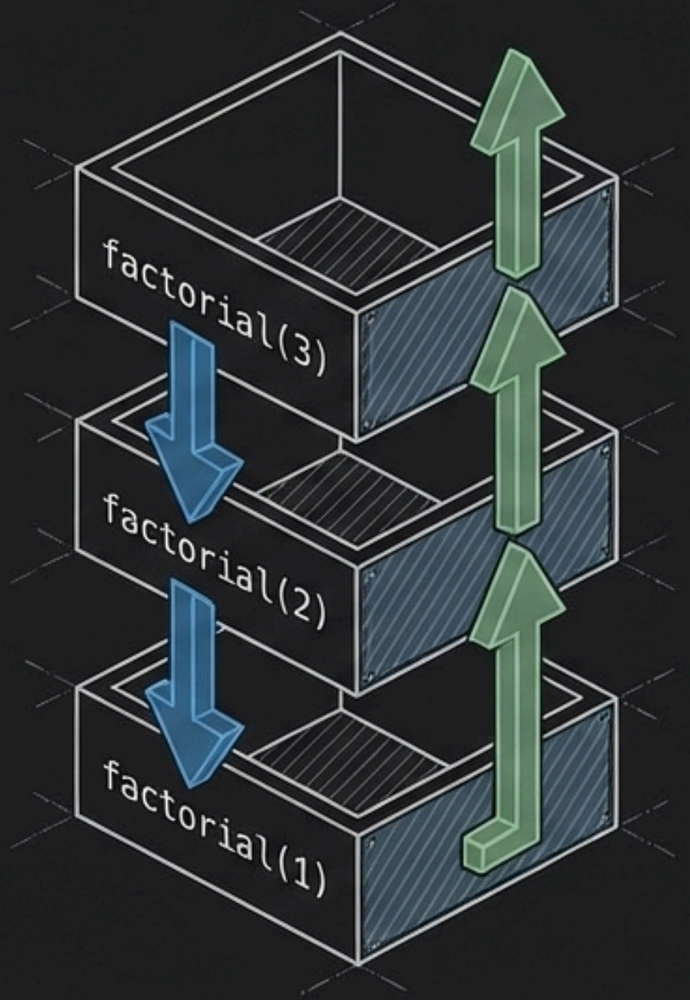
Pattern 8: Recursion & The Call Stack

Use when: Branching logic, subsets, tree traversals, backtracking.

The Logic: A function calling itself. It requires two strict rules: The Recursive Case (the mirror) and the Base Case (the exit).



The Mirror Inside a Mirror



The Diagnostic Matrix

How to recognize the optimal pattern instantly in a live interview based on problem constraints.

The Clue (Symptom)	The Pattern (Cure)	Expected Complexity
Sorted Array + Target Pair	Two Pointers	`O(n) Time, O(1) Space`
Contiguous Subarray Max/Min	Sliding Window	`O(n) Time, O(1) Space`
Multiple Range Queries	Prefix Sum	`O(1) Query Time, O(n) Space`
Max Subarray Sum (Positives & Negatives)	Kadane's Algorithm	`O(n) Time, O(1) Space`
String Frequencies / Duplicates	HashMap / Frequency Map	`O(n) Time, O(n) Space`
Sequence in Unsorted Array	HashSet Lookup	`O(n) Time, O(n) Space`

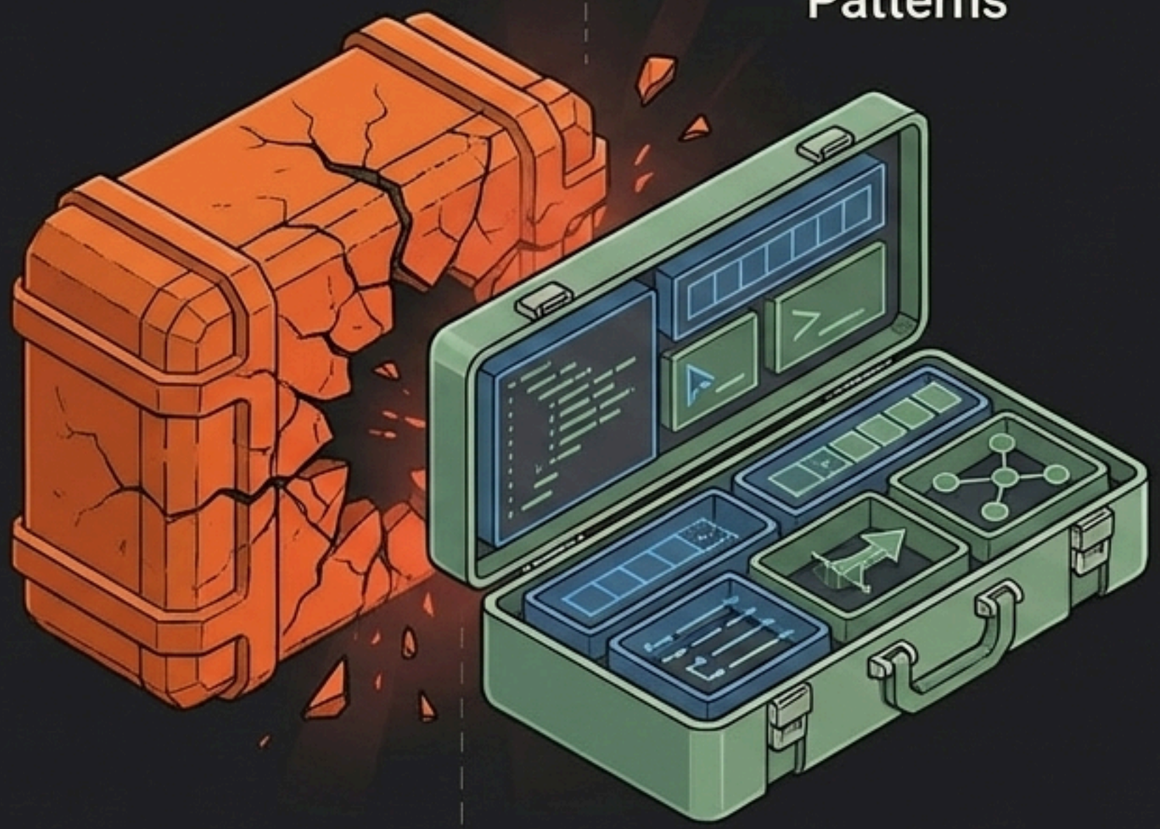
Write The Optimal Solution

DSA is not about memorizing 500 random problems. It is about learning how to think like an engineer.

Programming tells the computer what to do. Data Structures and Algorithms tell the computer how to do it efficiently.

$O(n^2)$ Brute Force

O(n) and O(log n) Patterns



Don’t rush to code.
Recognize the pattern.